

Scouts ML Automation Pipeline: n8n Workflow Documentation

Project: Scouts Tunisia DW

Workflow Files: `n8n_scouts_automation.json`

This document details the ML workflow architecture designed in **n8n**, structured entirely to map to the exact requirements provided in the Rubric.

A — Workflow Design (ML Pipeline Architecture)

The pipeline is modular, perfectly linear, and handles the complete ML lifecycle literally mirroring the rubric's required architecture: **Trigger -> Data Retrieval -> Model Execution -> Output Storage**

1. **Trigger (Webhook Node):** Acts as the event-driven initiator for inference logic.
2. **Data Retrieval (HTTP Request Node):** Pings the backend to extract the fresh state from the Data Warehouse layer.
3. **Model Execution (HTTP Request Node):** Triggers the Flask API (`/api/obj/1`) to run the active Random Forest regression models on the retrieved data block.
4. **Output Storage (Write to File Node):** Automatically streams the `json.body` responses directly into `inference_results.json` locally.

The nodes are heavily labelled inside the exported `n8n_scouts_automation.json` with descriptive tags and visual inline notes detailing their role in the sequence.

B — ML Model Integration (Technical Implementation)

The n8n workflow successfully integrates with the existing ML stack.

- **Full Integration:** The ML model is hosted inside a deployed Flask API backend (`app.py`), while retraining logic is encapsulated in `run_all_ml.py`.
 - **Varied Node Usage:** The design integrates standard external API calls and deeper local OS integrations:
 1. **Webhook Node** and **HTTP Request nodes** successfully manage the API inputs & outputs end-to-end.
 2. **Cron (Schedule) Node** manages execution scheduling.
 3. **Execute Command Node** manages OS-level integration (`python run_all_ml.py`).
-

C — Automation Logic (Inference / Retraining)

The workflow handles both ad-hoc event inference and scheduled lifecycle retraining without manual intervention.

- **Automated Inference Pipeline:** Governed strictly by the Webhook trigger. Once the event occurs, predictions are automatically grabbed and pipelined out to the Storage Node automatically.
 - **Retraining Automation:** Running parallel to inference, a **Cron Trigger** node fires automatically at 2:00 AM every night. It routes directly to the **Automated Retraining (Execute Command)** node, causing `run_all_ml.py` to pull fresh DW records to prevent model drift.
-

D — Robustness & Monitoring (Error Handling & Logs)

The architecture is built to gracefully handle server downtime and pipeline crashes.

- **Error Handling & Notifications:** The n8n layout includes a global **Error Catch (Error Trigger)** node intercepting any pipeline failures (like API downtime or Script tracebacks).
- **Alert Structure:** If a node fails, execution is immediately routed to the **Notifications (Slack)** node. The payload automatically pushes `{{$.json.execution.error.message}}` to the data engineering team's channel for rapid triage.

Workflow is ready for direct native import using the "Import from File" feature in the n8n UI.